Engineering Sketch Pad (ESP)



ESP Tutorial

John F. Dannenhoffer, III

Geocentric Technologies LLC

john@geocentrictech.com

Syracuse University

jfdannen@syr.edu

For ESP v1.29+

Overview

- ESP Overview
  - Background and Objectives
  - ESP Architecture
  - Distinguishing Features
- Using ESP to generate models
- Availability

Background (1)

- Over the past 40 years, there have been an increasingly-complex (complicated) series of "CAD" systems to support the geometry needs of the manufacturers of mechanical devices
  - CAD = "computer aided drafting"
  - CAD = "computer-aided drawing"
  - CAD = "computer-aided design"
  - CAD = "computer-aided development"
- "CAD" has sometimes been erroneously equated with geometry

Background (2)

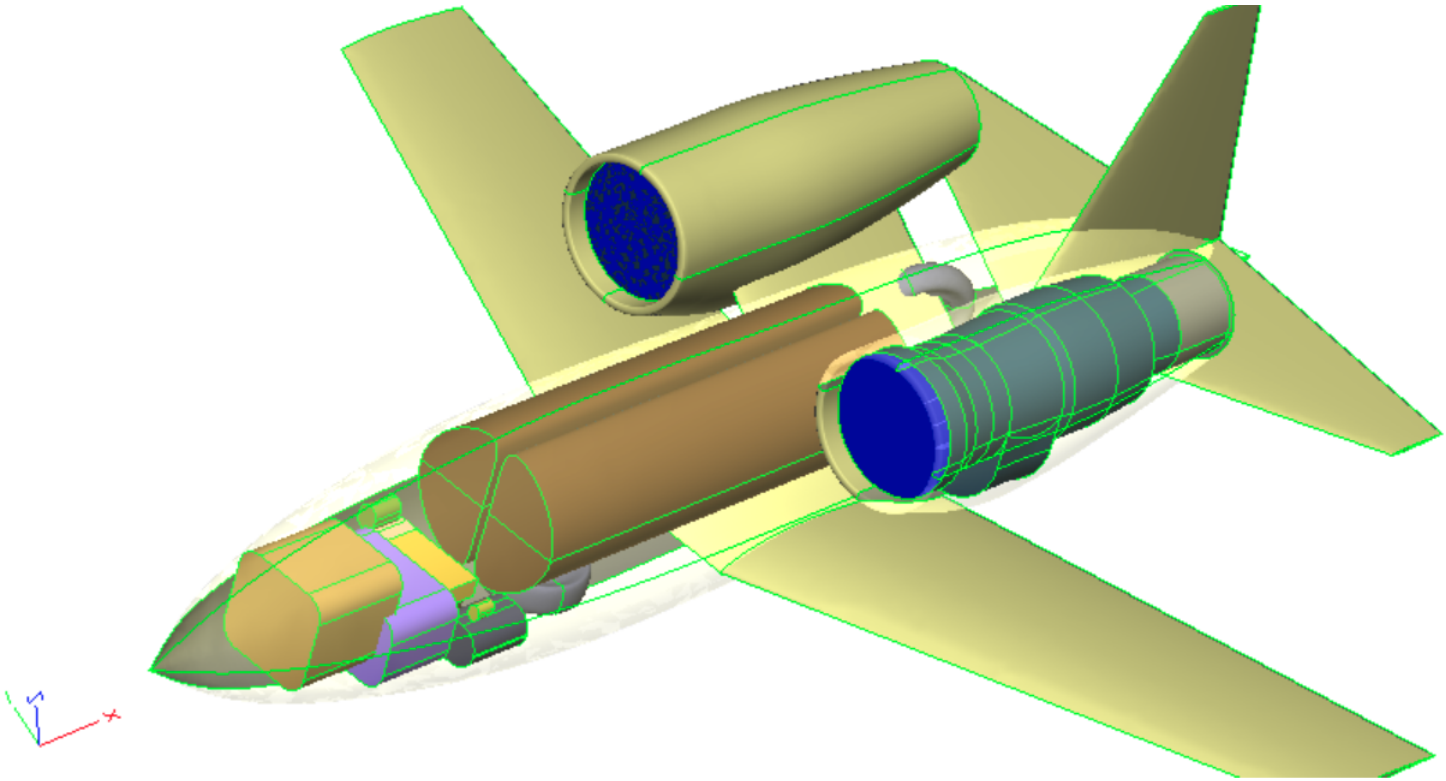
- These systems are built around the notion that the developer of a geometric model should construct the model to be consistent with the manufacturing process (**mCAD**)
- The analytical designer of a system wants to think about the function and performance of the device being generated, often leading to the generation of a separate **aCAD** model
- The modeling techniques supported by **aCAD** and **mCAD** are often so dissimilar that model transfer between them is done by limited translators or by "starting over"
- This one-way path from **aCAD** to **mCAD** leads to a "broken process"

Objective

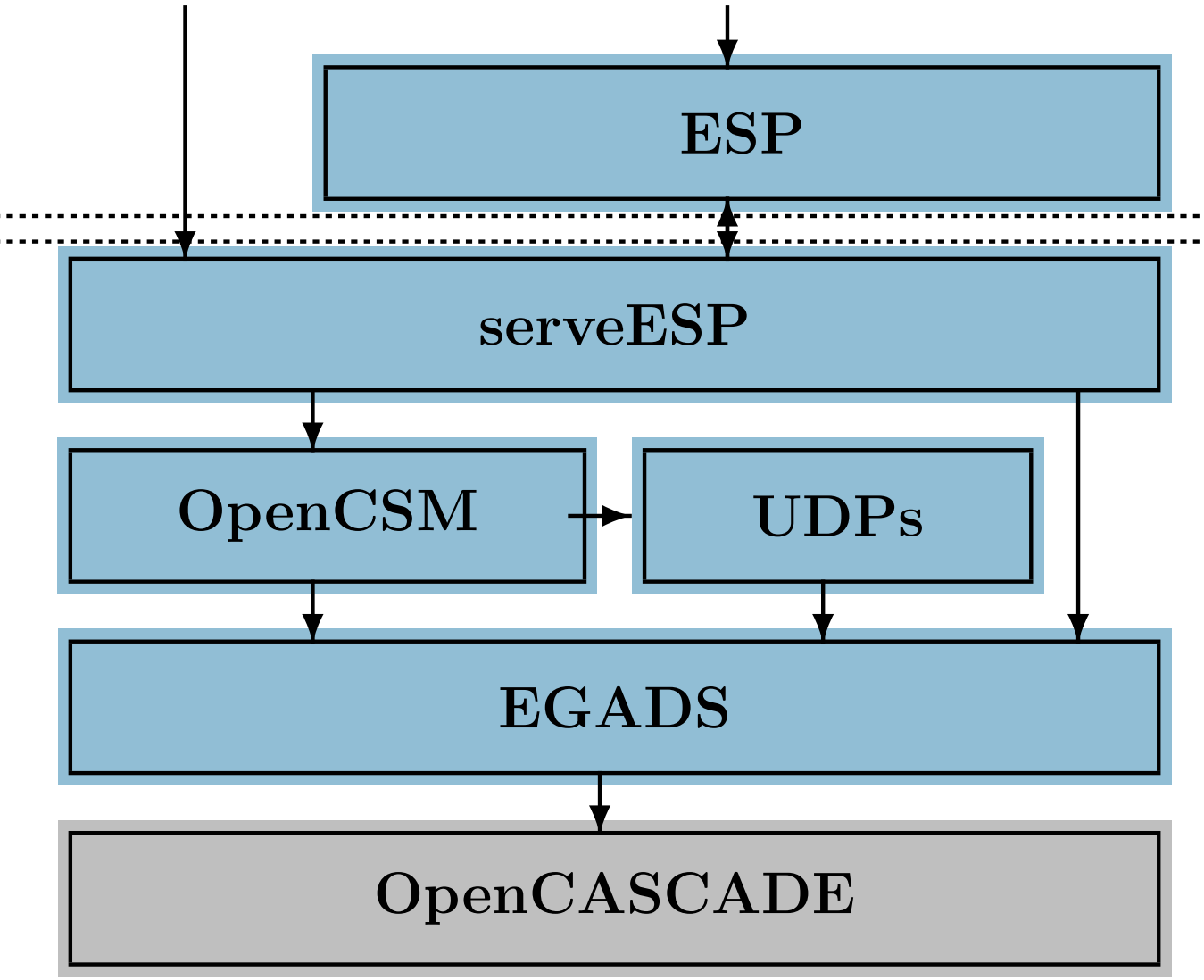
- ESP is:
  - a geometry creation and manipulation system designed specifically to support the analysis and design of aerospace vehicles
  - can be run stand-alone for the development of models
  - can be embedded into other analysis and design systems to support their geometry needs
- ESP is not:
  - a full-featured computer-aided design (CAD) system designed specifically to support the mechanical design and manufacturing of any complex system
  - a system to be used for creating “drawings”

Example ESP Model

Built in Advanced-User Training



ESP Architecture



user interactions

system driver
create graphic objects

design parameters
feature tree
user-defined primitives
sensitivities
top-down & bottom-up
persistent attribution
watertight tessellation

geometry primitives
boundary representations

Distinguishing Features — Solid Modeler

- Construction process guarantees that models are realizable solids
  - watertight representation needed for grid generators
  - sheets and wires are supported when needed
- Parametric models are defined in terms of:
  - Feature Tree
    - "recipe" for how to construct the configuration
  - Design Parameters
    - "values" that describe any particular instance of the configuration

Distinguishing Features — Feature-based

- Configurations start with the generation of primitives
  - standard primitives: point, box, sphere, cone, cylinder, torus
  - grown primitives (from sketches): extrude, rule, blend, revolve, sweep, loft
  - user-defined primitives (UDPs)
- Bodys can be modified
  - transformations: translate, rotate, scale, mirror
  - applications: fillet, chamfer, hollow
- Bodys can be combined
  - Booleans: intersect, subtract, union
  - other: join, connect, extract, elevate

Construction Process (1)

```
      # bolt example

      # design parameters
 1:   DESPMTR   Thead     1.00   # thickness of head
 2:   DESPMTR   Whead     3.00   # width     of head
 3:   DESPMTR   Fhead     0.50   # fraction  of head that is flat

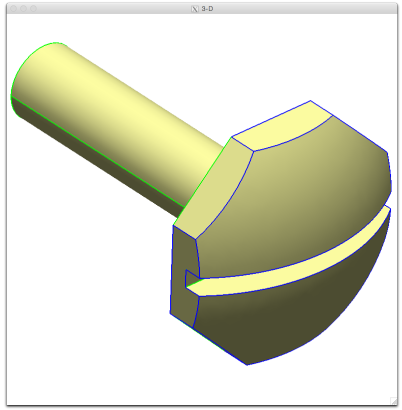
 4:   DESPMTR   Dslot     0.75   # depth of slot
 5:   DESPMTR   Wslot     0.25   # width of slot

 6:   DESPMTR   Lshaft    4.00   # length   of shaft
 7:   DESPMTR   Dshaft    1.00   # diameter of shaft

 8:   DESPMTR   sfact     0.50   # overall scale factor

      # head
 9:   BOX       0      -Whead/2  -Whead/2  Thead   Whead     Whead
10:   ROTATEX   90  0  0
11:   BOX       0      -Whead/2  -Whead/2  Thead   Whead     Whead
12:   ROTATEX   45  0  0
13:   INTERSECT

...
```



Construction Process (2)

```
...

14:   SET       Rhead   (Whead^2/4+(1-Fhead)^2*Thead^2)/(2*Thead*(1-Fhead))

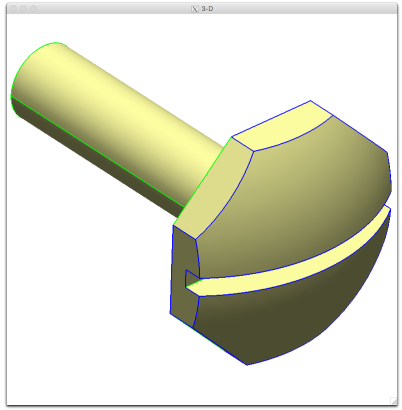
15:   SPHERE    0              0   0    Rhead
16:   TRANSLATE Thead-Rhead    0   0
17:   INTERSECT

      # slot
18:   BOX       Thead-Dslot  -Wslot/2   -Whead   2*Thead   Wslot    2*Whead
19:   SUBTRACT

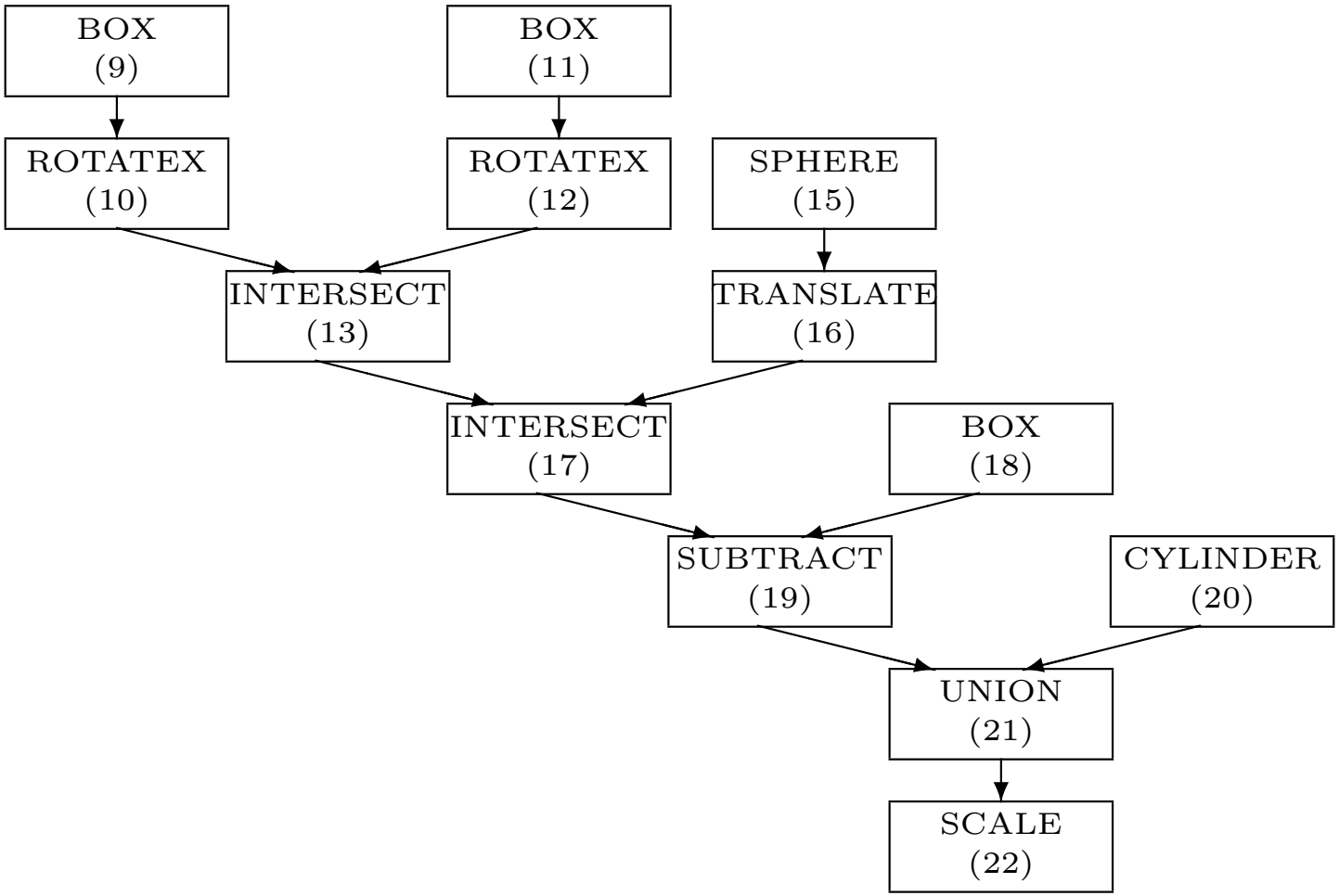
      # shaft
20:   CYLINDER  -Lshaft   0   0   0   0   0   Dshaft/2
21:   UNION

22:   SCALE     sfact

23:   END
```



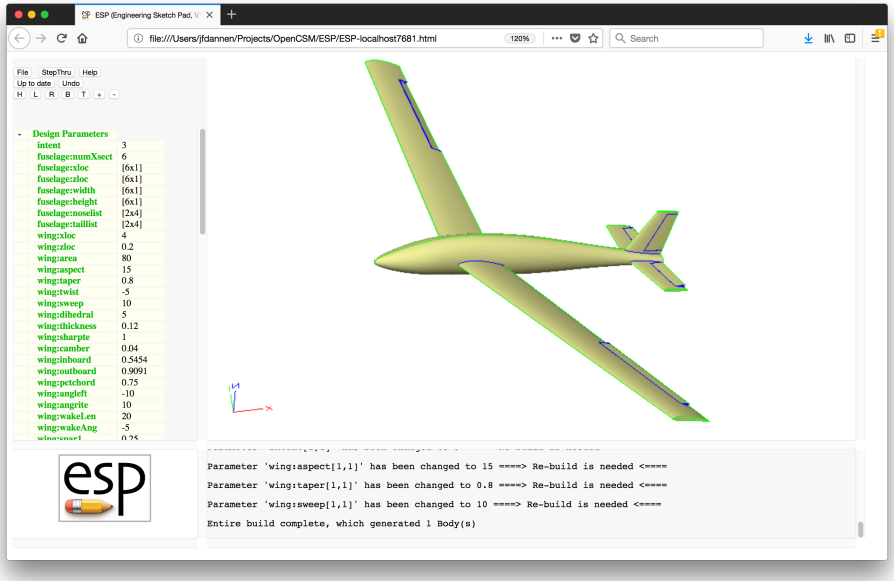
Review of Construction Process (3)



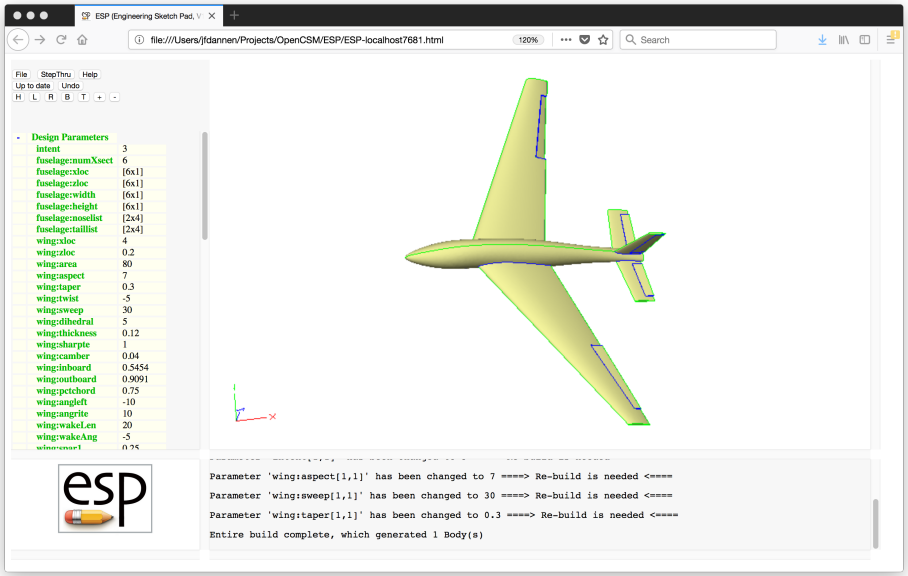
Distinguishing Features — Parametric

- ESP models typically contain one or more Design Parameters
- Design Parameters can be single-valued, 1D vectors, or 2D arrays of numbers
- Each Design Parameter has a current value, upper- and lower-bounds, and a current "velocity" (which is used to define sensitivities)
- Design Parameters can be "set" and "get"
  - through ESP's tree window
  - through Pyscript
  - externally via calls to the Application Programming Interface (API)
- Arguments of all operations can be written as "expressions" that reference Design Parameters

Parameter Changes for Glider



$$\begin{aligned} \text{aspect} &= 15 \\ \text{sweep} &= 10 \\ \text{taper} &= 0.8 \end{aligned}$$

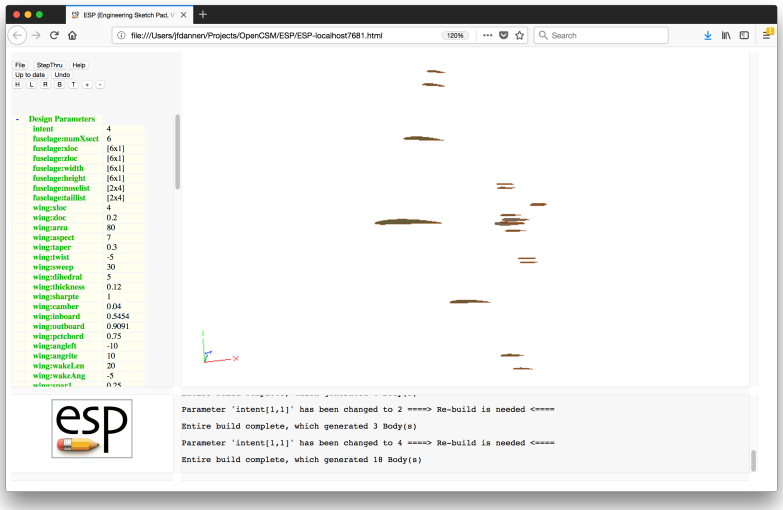


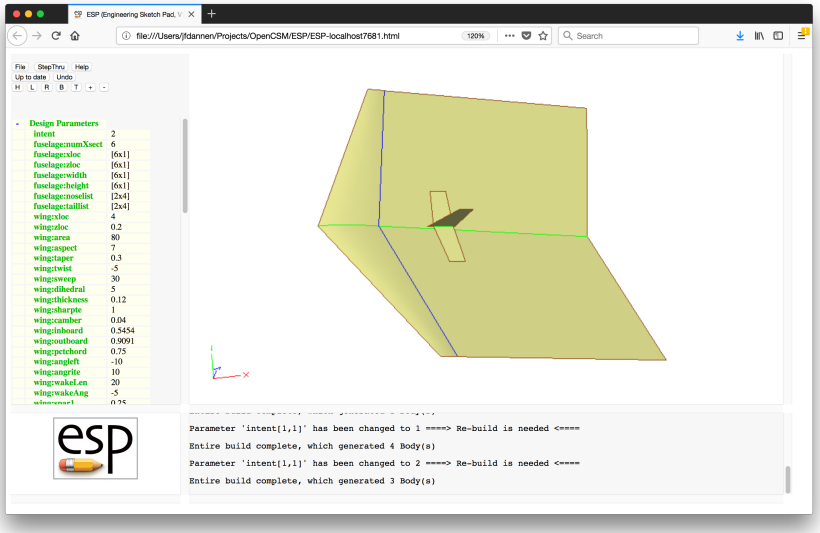
$$\begin{aligned} \text{aspect} &= 7 \\ \text{sweep} &= 30 \\ \text{taper} &= 0.3 \end{aligned}$$

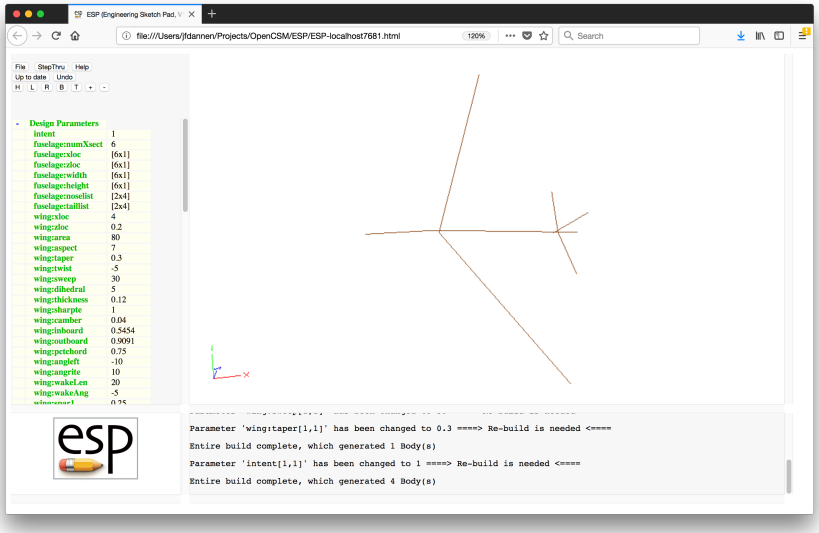
Distinguishing Features — Associative

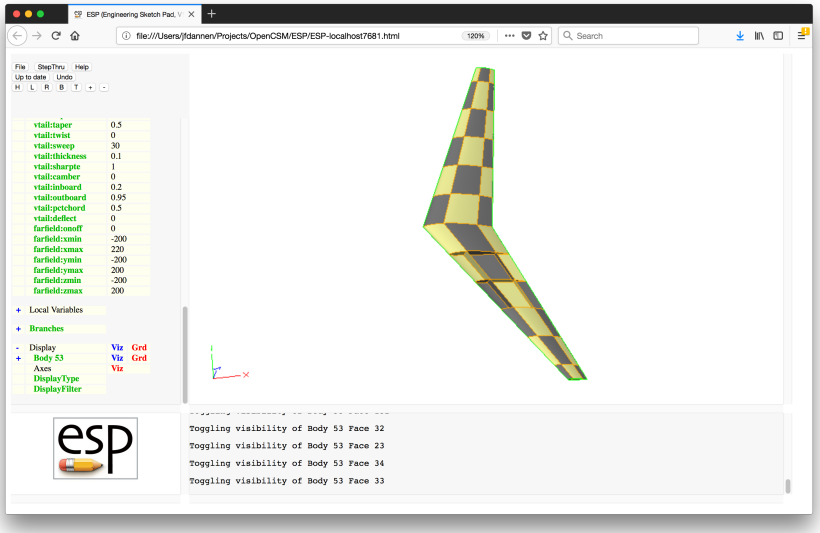
- ESP maintains a set of global and local attributes on a configuration that are persistent through rebuilds
- Supports the generation of multi-fidelity models
  - attributes can be used to associate conceptually-similar parts in the various models
- Supports the generation of multi-disciplinary models
  - attributes can be used to associate surface groups which share common loads and displacements
- Supports the "marking" of Faces and Edges with attributes such as boundary conditions, nominal grid spacings, material properties, ...

Multiple Models for Glider





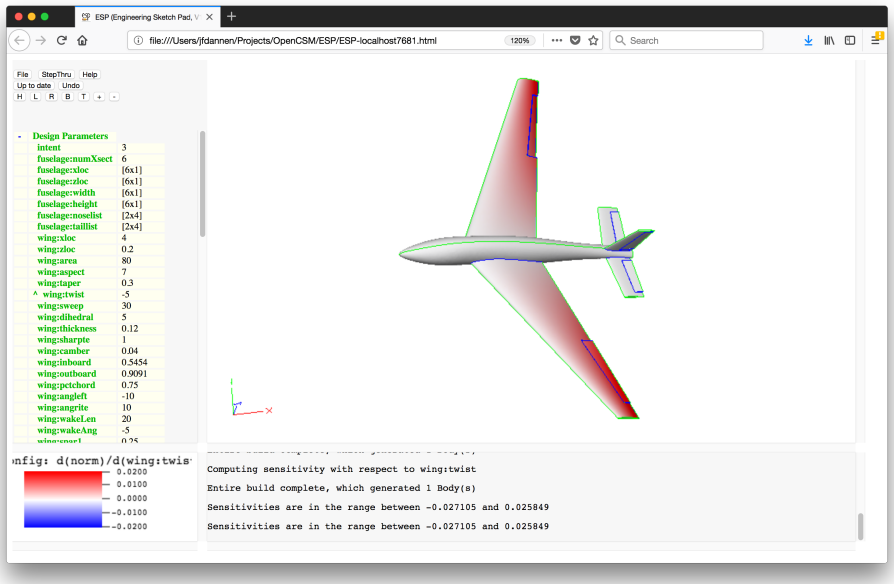




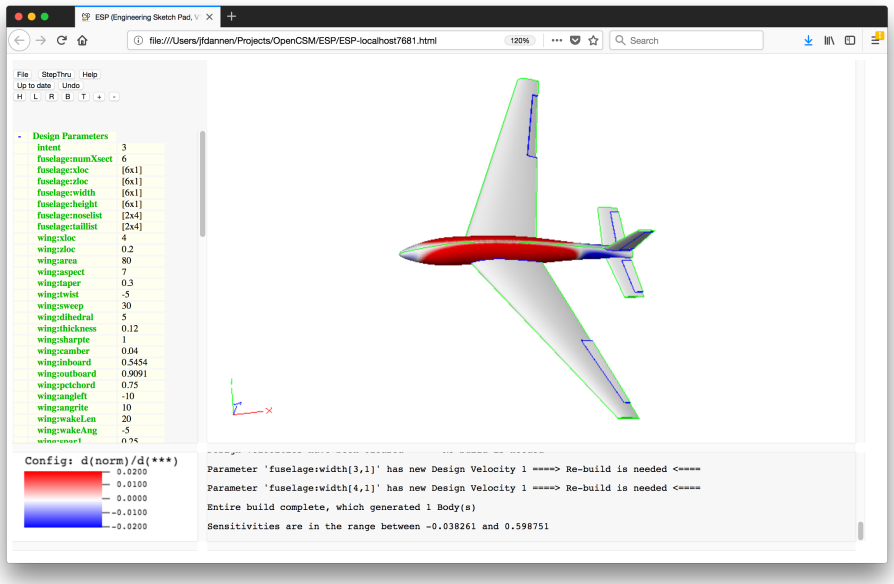
Distinguishing Features — Differentiated

- `ESP` allows a user to compute the sensitivity of any part of a configuration with respect to any Design Parameter
- Many of `OpenCSM`'s commands have been analytically "differentiated"
  - efficient, since there is no need to re-generate the configuration
  - accurate, since there is no truncation error associated with "differencing"
- Other commands (currently) require the use of finite-differenced sensitivities
  - robust, due to new mapping technique
  - less efficient, since it requires the generation of a "perturbed" configuration
  - less accurate, since one needs to carefully select a "perturbation step" that is a balance between truncation and round-off errors

Sensitivities for Glider



twist



fuselage width

Distinguishing Features — Extensible

- Users can add their own user-defined primitives (UDPs)
  - create a single primitive solid
  - are written in C, C++, or FORTRAN and are compiled
  - can be written either top-down or bottom-up
  - have access to the entire suite of methods provided by `EGADS`
  - are coupled into `ESP` dynamically at run time
- Users can add their own user-defined functions (UDFs)
  - consume one or more Bodys from stack
  - are otherwise similar to UDPs
- Users can add their own user-defined components (UDCs)
  - can be thought of as "macros"
  - create zero or more Bodys
  - are written as `.csm`-type scripts

Shipped User-defined Primitives

- bezier
- biconvex
- box
- bspline
- csm
- ellipse
- eqn2body
- fitcurve
- freeform
- hex
- import
- kulfan
- naca456
- naca
- nurbbody
- parabaloid
- parsec
- pod
- poly
- prop
- radwaf
- sample
- sew
- stag
- supell
- vsp3
- waffle
- warp

Shipped User-defined Functions

- catmull
- compare
- createBEM
- createPoly
- deform
- droop
- dumpPmtrs
- editAttr
- flend
- ganged
- guide
- linalg
- matchBodys
- mechanism
- naca6mv
- nacelle
- nuscale
- offset
- printBbox
- printBrep
- printEgo
- shadow
- slices
- stiffener

Distinguishing Features — Deployable

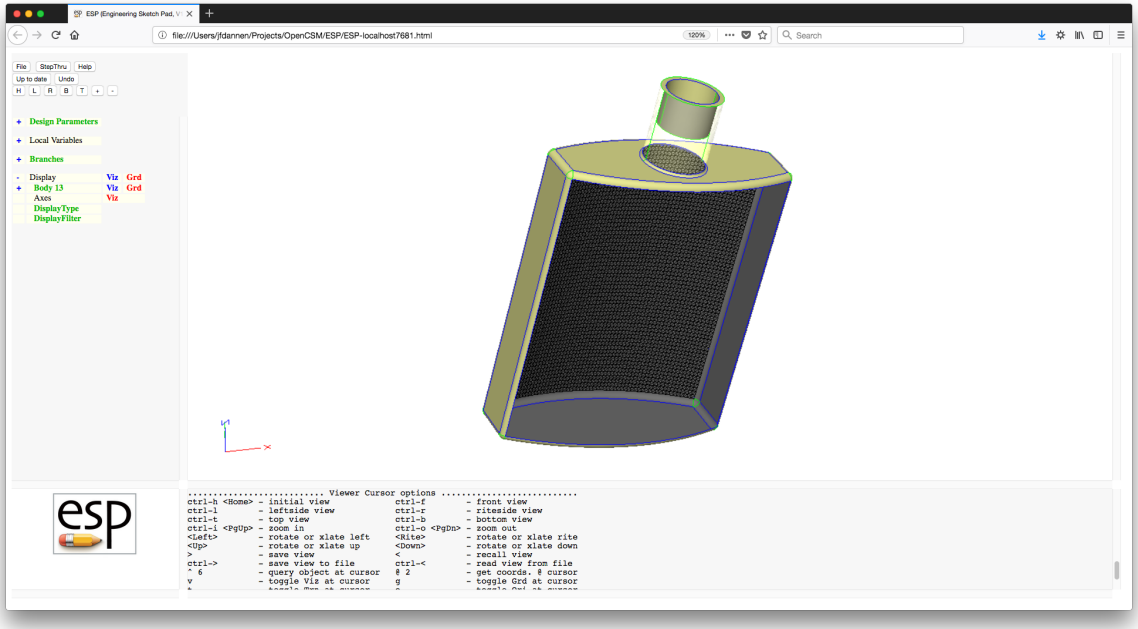
- ESP's back-end (server) runs on a wide variety of modern compute platforms
  - LINUX
  - MAC-OS
  - Windows
- ESP's user-interface (client) runs in most modern web browsers
  - Google Chrome
  - Safari
  - Edge
  - FireFox
- ESP can be distributed anywhere in the computer environment
  - open-source project (using the LGPL 2.1 license) that is distributed as source

Distinguishing Features — Embeddable

- Models are defined in `.csm` files
  - human readable ASCII
  - stack-like language that is consistent with Feature Tree traversal
  - contains looping via "patterns"
  - contains logical (if/then) constructs
  - contains error recovery via thrown/caught signals
- `OpenCSM` modeling system is defined by an Application Programming Interface (API) that allows it to be embedded into other applications
  - load a Master Model
  - interrogate and/or edit the Master Model
  - execute the Feature Tree and create BRep(s)
  - interrogate the BRep(s)
  - "set" and "get" sensitivities

ESP Screen Layout

- GraphicsWindow
  - 3D image
  - 2D sketcher
  - forms
- TreeWindow
  - Design Parameters
  - Local Variables
  - Branches
  - Display
- KeyWindow
  - color key
- MessageWindow



Creating a Script (1)

Using the ESP Interface

- Method:
  - start ESP: `serveESP`
  - add Design Parameter by pressing **DesignParameters**
  - add Branch by pressing **Branch**
- Advantages:
  - most similar to other CAD packages
  - can use interactive sketcher
- Disadvantages:
  - generally slow
  - cannot add comments, indentation, etc.
  - harder to debug

Creating a Script (2)

Using an External Text Editor

- Method:
  - use any text editor to create `myFile.csm`
  - run `ESP`: `serveESP -loadEgads -dumpEgads myFile`
- Advantages;
  - can use any editor with which you are familiar
  - easy to add comments, spacing, indentation, ...
- Disadvantages:
  - do not get help in writing `.csm` file
  - cannot use interactive sketcher (except via a UDC)
  - requires many `ESP` restarts

Creating a Script (3)

Using the Integrated Code Editor

- Method:
  - start ESP: `serveESP`
  - **File→Edit** and then **Save**
- Advantages:
  - context-sensitive editor with hints and links to Help
  - easy to add comments, spacing, indentation, ...
  - supports debugging and tracing
- Disadvantages:
  - slightly different key mappings
  - cannot use interactive sketcher (except via a UDC)

Collaboration Mode (1)

- Inspired by Pair Programming* paradigm
  - Driver: writes programs, detail-level, tactical decisions
  - Navigator: overlooks, feedback, high-level strategic choices
- Studies on pair programming show:
  - only about 20% increase in overall time
  - but about 5-fold decrease in error rate
- ESP uses a browser-based, client-server architecture
- Interface is similar for single as well as multiple users
- Interchangeable role of Driver and Navigator by "Passing the Ball"
- Interface can be coupled with voice and/or other visual tools to enhance experience

Collaboration Mode (2)

- New collaborative environment in `ESP` has several benefits:
  - shared ownership of the model
  - tendency to take fewer short-cuts
  - low error rate
  - reduced labor is more apparent while performing complex tasks
- To use:
  - first user starts `serveESP` as usual (with a `hostname` that is accessible to all expected users)
  - subsequent users start a browser on `ESP.html` and use same `hostname:port` as first user

Other ESP-related Capabilities

- `Pyscript` — integrated Python interpreter
- `Miniopt` — non-linear constrained optimizer
- `Vsp3` — interface to OpenVSP models (with sensitivities)
- `Plugs` — parametric geometry from a cloud of points
- `Slugs` — static geometry from a cloud of points
- `EGADS-lite` — parallel geometry for HPC environments
- `CAPS` — integration with analyses and optimization

Integration with Analyses (CAPS)

- Project Objective:
  - establish a computational geometry, meshing, and analysis model generation tool that will enable collaboration between conceptual design, multidisciplinary optimization, and high fidelity simulation efforts.
  - provide tools & techniques for generalizing analysis coupling
    - multi-disciplinary coupling: aeroelastic, FSI
    - multi-fidelity coupling: conceptual and preliminary design
  - provide the tools & techniques for rigorously dealing with geometry in a design framework/process
  - be flexible enough to handle almost any aerospace configuration

CAPS Design Principles

- A user should only have to define a configuration once
  - should be driven by user-defined Design Parameters
  - should be attributed (tagged) during the construction process
- Must support multi-fidelity and multi-disciplinary analyses
  - analysis tools should not have to be modified
  - system must be expandable so that new analyses can be added to the system at any time
- CAPS should handle mechanics of transforming the user's configuration into a "view" of the configuration that is needed by the analysis tool
- System execution must be flexible enough to support nearly any design or analysis task

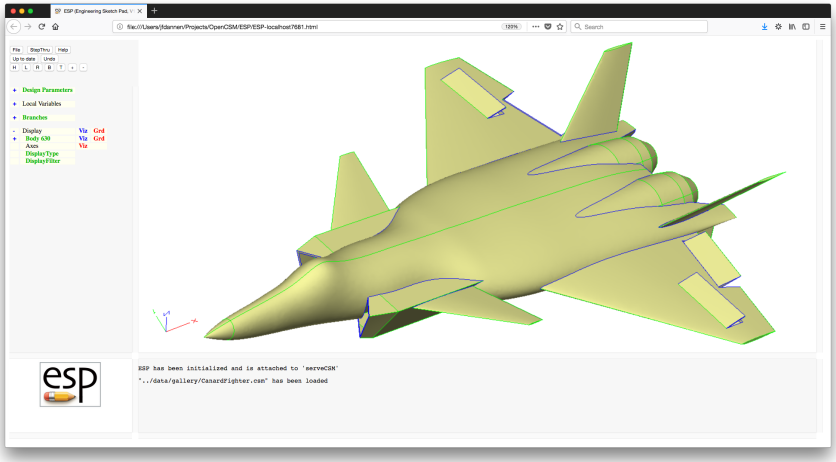
Analysis Interfaces Shipped with CAPS

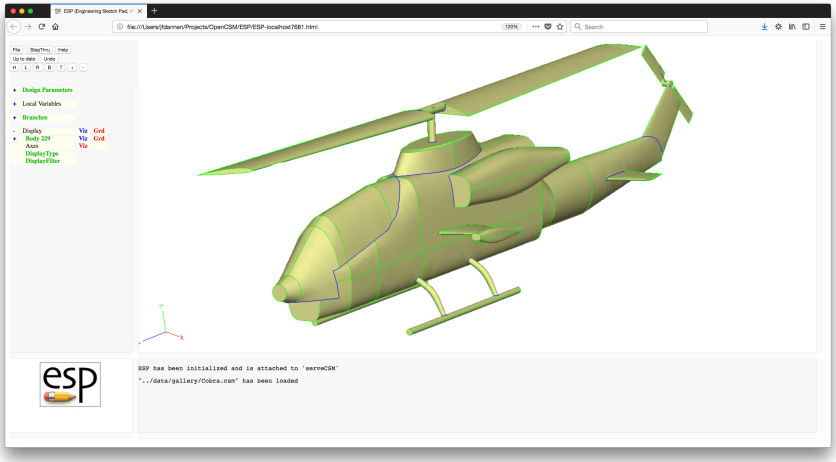
- Surface Meshing
  - EGADS
  - AFLR4
  - Delaundo
  - Pointwise
- Volume Meshing
  - TetGen
  - AFLR3
  - Pointwise
- Aerodynamics
  - FRICTION
  - AWAVE
  - XFOIL
  - TSFOIL
  - AVL
  - CART3D
  - SU2
  - FUN3D
- Structures
  - MYSTRAN
  - NASTRAN
  - Astros

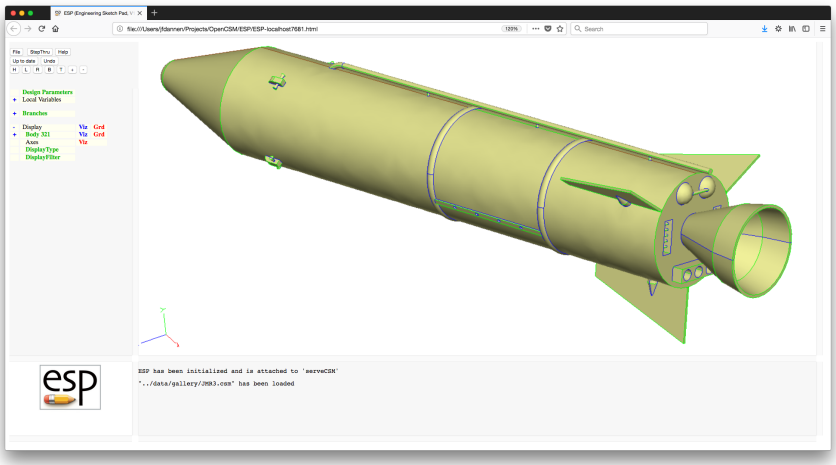
Using ESP to Create Models

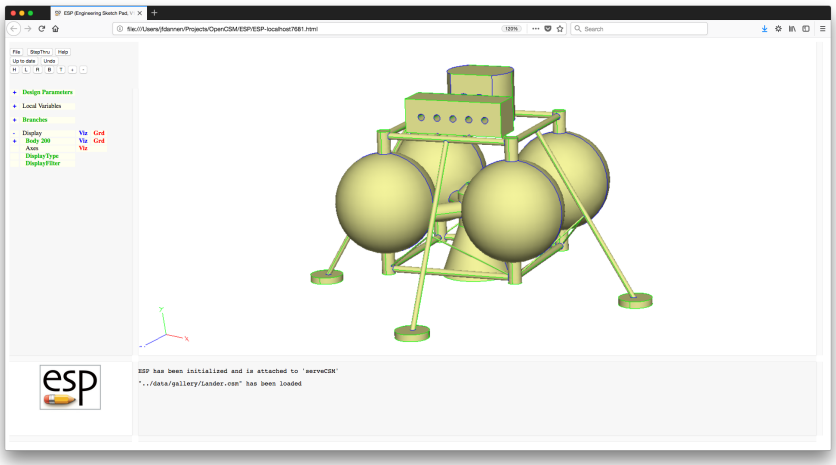
- Use cases:
  - build a configuration from scratch
    - write script, possibly using aerospace-specific user-defined primitives
  - add features to an existing model
    - configuration can be imported via an IGES or STEP file
    - standard ESP featured used to model supports, wind-tunnel walls, etc.
- save model in an `.egads` file, which is readable by the Chimera Grid Tools

ESP Models Built from Scratch



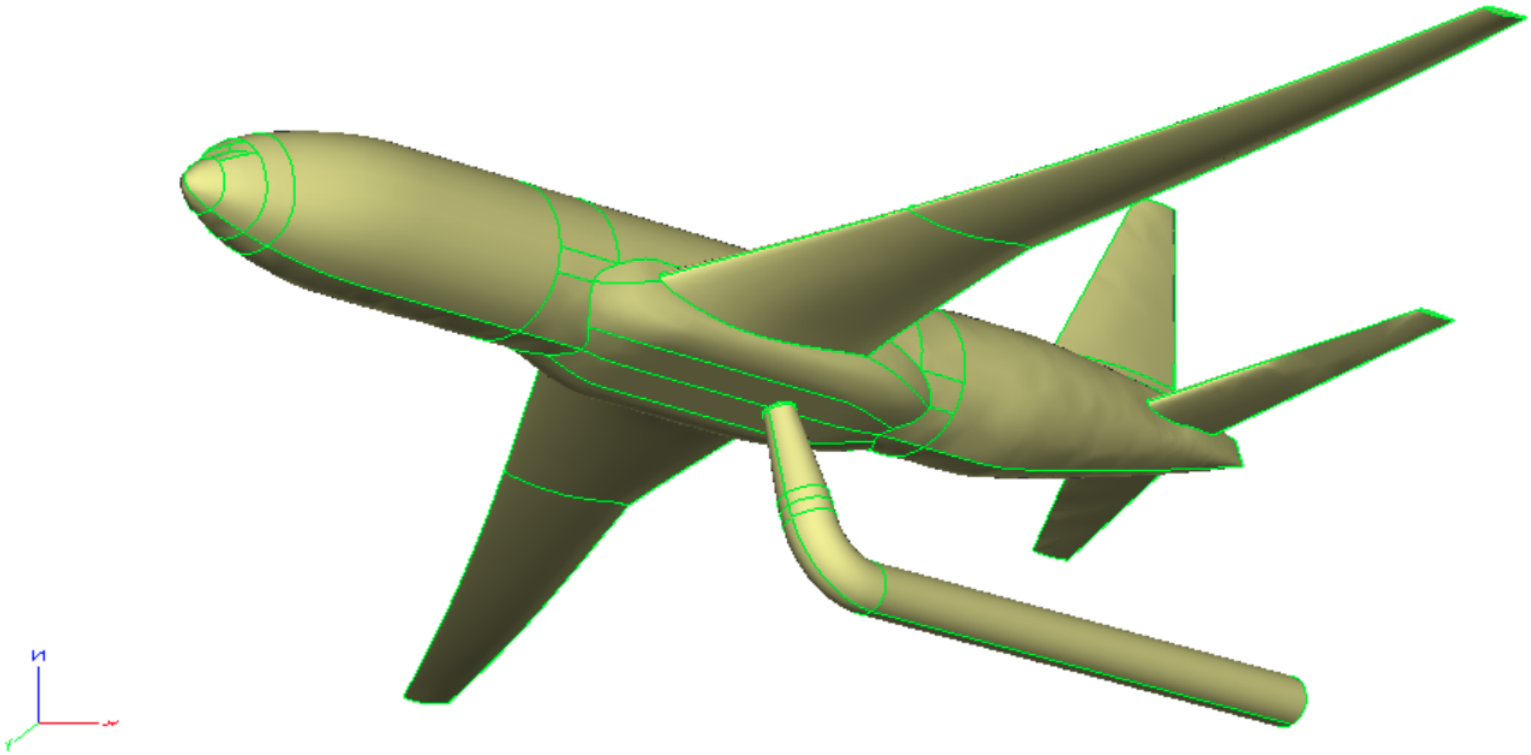






ESP Model by Modifying a STEP File

Base model is Common Research Model



Availability

- Currently hosted at `acdl.mit.edu/ESP`
- Will soon be hosted at `geocentrictech.com`
- Websites include:
  - source tarball
  - pre-built versions for Linux, MacOS, and Windows
  - training tarball
  - `ESP` publications
  - dependencies: Python and OpenCASCADE
  - archive of all previous versions